The Architecture of Opportunity

Engineering a Deterministic Data Pipeline for Market-Aware Content Ideation

Colombini Francesco[*], **Nucci Nicolò**[†]

Data Management Project

Abstract. The creator economy suffers from fundamental information asymmetry: platforms provide abundant retrospective analytics but negligible predictive support. This work addresses a critical question: *can subjective content ideation be replaced by deterministic data pipelines?* A three-tier local architecture has been engineered to synthesize search intent signals (Demand) with empirically validated performance metrics (Supply). Through rigorous ETL enforcement and statistical quality constraints, we demonstrate that creative ideation can be treated as a data engineering challenge where generative outputs are constrained by statistical evidence rather than plausible speculation.

1 Introduction

Content creation on YouTube operates in an algorithmically ranked environment where thousands of videos compete simultaneously for visibility. The central challenge isn't *how* to produce content, but *what* to produce and *how* to position it for maximum discoverability. Despite extensive post-publication analytics, platforms offer minimal support during the **ideation phase**, which represents the critical pre-publication decision point. Creators receive historical data about past performance but lack systematic guidance for selecting titles, tags, keywords or publication timing based on observable market conditions. The "Cold Start Problem" manifests as strategic paralysis: without prior performance data, creators resort to intuition or imitation. This asymmetry creates a bottleneck where data-rich platforms fail to translate retrospective insights into prospective strategy.

1.1 Research Objective

This project designs a data-driven interface supporting content ideation *before* publication. The system integrates existing video supply on YouTube with user demand signals from Google Trends, generating structured suggestions for titles, tags, descriptive keywords and publication windows. The goal is to assist creators in designing content aligned with current demand patterns, increasing visibility probability within competitive landscapes.

2 System Architecture

The architecture follows a **three-tier ETL model** with clear separation between data persistence and analytical reasoning. The first layer handles data ingestion and staging, retrieving raw data from external APIs and loading it directly into a relational database staging area without modification. The second layer manages data transformation and warehousing, performing cleaning, validation and structuring via SQL logic to create analytical models. The third layer executes advanced analytics and LLM-based reasoning using curated warehouse data. Finally, a user interface layer orchestrates execution flow and manages user interaction. To model this data, the backend utilizes two complementary Dimensional Fact Models linked by a shared Keyword Dimension. This design logically separates Content Supply performance metrics from Market Demand, ensuring that these independent variables are correlated only during the analytical phase without structural coupling in the storage layer.

3 Data Acquisition Layer

The acquisition layer represents the foundational tier where external signals are captured and persisted. This phase focuses on maximizing data completeness while respecting platform constraints and maintaining reproducibility.

3.1 YouTube: API-Driven Discovery

YouTube data acquisition is implemented through an active, API-driven pipeline designed to address three fundamental challenges: daily quota limitations (10,000 units per API key), result set truncation (maximum 500 results per query configuration), and ranking instability across pagination. The system adopts a multi-layered strategy that balances recall maximization with operational stability and reproducibility.

3.1.1 Quota Resilience via Key Rotation

The YouTube Data API enforces strict daily quotas that make naive large-scale queries impractical. To ensure continuity of execution, the acquisition logic implements an API key rotation mechanism. The system maintains an in-memory state dictionary tracking the current position in the multi-dimensional discovery space. When quota exhaustion is detected, the current state is preserved and execution resumes seamlessly using the next available API key.

The retry mechanism distinguishes between transient failures, such as network timeouts or temporary rate limit-

ing—which trigger exponential backoff—and structural failures such as quota exhaustion, which trigger immediate key rotation. This design prevents premature termination of the acquisition process while respecting API constraints, effectively transforming hard quota limits into a manageable scaling parameter.

3.1.2 Multi-Dimensional Discovery Strategy

The discovery phase aims to identify relevant video identifiers and capture their associated contextual metadata, rather than to retrieve full engagement statistics. Each query produces a stream of lightweight *discovery events* containing unique video identifiers together with minimal snippet-level information (title, description, publication timestamp, and channel identifiers). Each event is further enriched with structured provenance metadata describing how the video was discovered, including the seed keyword, keyword variant, ranking order, and temporal window boundaries. Redundancy is intentionally preserved at this stage, allowing the same video to be observed under multiple query contexts to maximize recall.
The discovery space is systematically explored across two orthogonal dimensions.

Keyword Variant Generation To account for formatting variability in video metadata, the system generates multiple textual variants for keywords (e.g., *machine learning* is searched also as *machinelearning* and *machine-learning*). For single-word keywords, the normalized keyword is used directly without variant expansion. The variant generation function is explicitly limited to two-word keywords in this implementation but can be improved; multi-word phrases beyond two tokens are truncated to their first two words, a constraint that must be considered when selecting seed keywords.

Temporal Window Decomposition Rather than querying the entire lookback period in a single request, the timeline is decomposed into smaller, non-overlapping temporal windows of configurable length. This approach multiplies effective search capacity, reduces ranking volatility due to real-time engagement dynamics, and enables granular resume points. The system defaults to daily granularity (one-day windows) but allows users to configure coarser resolutions to balance temporal coverage against API quota consumption.

3.1.3 Batch Metadata Retrieval

Following discovery, the pipeline performs an additional acquisition stage to retrieve detailed metadata associated with the discovered video identifiers. Video IDs are processed in batches of 50, the maximum size supported by the YouTube Data API, to maximize quota efficiency. For each batch, the system retrieves video-level statistics, including view counts, likes, comments, favorites, and content duration. Video durations are parsed using custom regular-expression–based extractors and converted into total seconds, enabling reliable classification of content format; videos shorter than 60 seconds are labeled as short-form content (Reels/Shorts). In the same way, unique channel identifiers are extracted and used to retrieve aggregated channel-level metadata, including subscriber counts (when publicly available), total uploaded video counts, cumulative channel views, and privacy-related flags. These statistics provide the contextual basis for subsequent normalization of video performance relative to channel size, allowing algorithmic favorability to be distinguished from effects driven purely by audience scale.

3.1.4 Structured Output via NDJSON

All discovery events and associated raw metadata are immediately serialized to disk using the Newline Delimited JSON (NDJSON) format, which acts as an intermediate, append-only buffer. This format supports line-by-line validation, ensuring that corrupted records do not invalidate the entire dataset, and enables robust recovery from execution interruptions or API quota exhaustion. Previously collected records remain persistently stored on disk, allowing the pipeline to resume execution without repeating expensive discovery operations.

3.1.5 Deduplication and Result Consolidation

The multi-dimensional discovery strategy intentionally introduces redundancy, as the same video may be retrieved multiple times through different keyword variants, temporal windows, or ranking orders. All discovery events are preserved during acquisition in order to retain full provenance information. Deduplication is deferred to subsequent processing stages, where global visibility over the complete set of discovery events is available. During consolidation, multiple occurrences of the same video identifier are resolved by retaining the first observed instance as the canonical representation, while subsequent duplicates are discarded. This *first-seen* strategy preserves the earliest discovery context while ensuring that downstream analysis operates on a clean, non-redundant dataset.

3.2 Google Trends: Semi-Manual Ingestion

Search intent data follows a semi-manual ingestion path designed to balance data quality with platform compliance. Users access the Google Trends web interface `https://trends.google.it/explore`, enter their target keyword and manually select temporal windows (7, 14, 30, 60 days), downloading both Top and Rising queries as CSV files.
This manual workflow is deliberate. Google Trends lacks an official API and automated scraping violates the platform's Terms of Service, risking IP bans and unreliable data. Manual exports from the official interface guarantee canonical formatting and give users explicit control over temporal coverage, ensuring alignment with YouTube data collection periods.
Once downloaded, CSV files are placed in a designated

landing directory where the ingestion function scans recursively. The system extracts metadata from filenames using regex patterns, expecting a format that encodes query type (top/rising), country code and temporal boundaries. These metadata components are injected as new columns during ingestion while preserving original CSV data for full traceability.

The function aggregates matching files into unified DataFrames for Top and Rising queries separately. Files that don't match the expected pattern are silently skipped, preventing failures from unrelated directory contents. Empty result sets produce empty DataFrames rather than errors, gracefully handling partial datasets. A critical limitation exists: the system cannot automatically verify keyword matching between Google Trends and YouTube data, as keywords aren't embedded in CSV filenames. Users must manually ensure consistency, as a mismatch won't be detected and will silently corrupt cross-dataset comparisons(e.g., downloading "machine learning" trends while analyzing "data science" videos).

3.3 Relational Storage and Topic Isolation

Despite semi-structured upstream formats (JSON for YouTube, CSV for Google Trends), the system adopts relational storage via SQL databases. This architectural choice is driven by several considerations.

SQL queries provide explicit, auditable representations of transformations, ensuring every aggregation, filter or join is a readable operation rather than an opaque in-memory manipulation.

Relational databases with typed columns consume significantly less disk space. Most importantly, SQL-based storage integrates universally with BI tools and data platforms, enabling downstream consumption.

The system implements a topic-first naming strategy, prefixing all tables with normalized keyword identifiers (e.g., `{topic}_youtube_videos`, `{topic}_google_trends_rising`). This provides multi-topic isolation, preventing namespace collisions when analyzing different keywords, logical organization that groups related tables by prefix, support for incremental updates without full database rebuilds and the ability to selectively delete topic-specific data without affecting other analyses.

The complete acquisition workflow orchestrates YouTube discovery (key loading, multi-dimensional search, batch statistics retrieval, duplicate reporting), Google Trends ingestion (CSV discovery, metadata extraction, temporal validation), NDJSON-to-DataFrame transformation (streaming, deduplication, schema alignment) and relational persistence (topic-prefixed table creation, bulk insertion, row count reporting). The system returns a dictionary of row counts, providing immediate feedback on acquisition success and enabling data quality monitoring.

4 Data Quality and Feature Engineering

After data acquisition and persistence, the system enters a structured data quality phase implemented entirely at the SQL level. This phase is responsible for validating raw inputs, enforcing consistency constraints and producing the canonical analytical datasets used in downstream scoring and reasoning. The quality layer is intentionally split into two parallel validation pipelines, one dedicated to YouTube platform data and one dedicated to Google Trends search data, reflecting the fundamentally different semantics, error modes and decision roles of the two sources.

4.1 YouTube Quality and Integration

YouTube data is inherently heterogeneous, combining video-level statistics, channel-level metadata and temporal attributes. The validation process addresses structural anomalies, statistical significance and metric derivation.

Before any analytical processing occurs, raw tables undergo strict validation. The system identifies videos with incomplete or logically inconsistent metadata through multi-condition filtering. Records are flagged when critical fields (`view_count`, `like_count`, `comment_count`, `duration_seconds`) contain NULL values, negative values or exhibit impossible states such as `like_count > view_count`, a pattern indicative of API synchronization errors or bot manipulation.

Channel-level validation isolates channels with missing subscriber counts or explicitly hidden subscriber information. These incomplete records are moved to dedicated quarantine tables (suffixed with `_incomplete`) rather than being deleted outright. This isolation architecture maintains full audit trails while preserving data integrity in the master dataset, enabling post-hoc investigation of anomalies without contaminating analytical results.

To eliminate noise from statistically insignificant samples, videos with fewer than 50 views are excluded from the analytical set. This threshold prevents artificial inflation of engagement ratios; for instance, a single view and single like would yield a misleading 100% engagement rate. The final clean dataset (`{topic}_youtube_full_statistics`) represents the union of valid video metadata, video statistics and channel statistics, ensuring every record contains complete and logically consistent information.

4.2 Feature Engineering

Once sanitized, the data is enriched with derived metrics calculated directly via SQL to maximize computational efficiency. These engineered features transform raw counts into normalized performance indicators that enable fair cross-video comparisons.

Engagement Rate (ER) is computed as:

$$ER_v = \frac{L_v + C_v}{V_v}$$

```sql
UPDATE {YT_FULL_STATS}
SET engagement_rate = CASE
    WHEN video_view_count > 0
    THEN CAST(like_count + comment_count
              AS REAL) / video_view_count
    ELSE NULL
END
```

This metric isolates algorithmic favourability independent of channel size. A value greater than 1.0 indicates the video reached beyond the channel's subscriber base, suggesting strong algorithmic amplification or viral potential. Conversely, values below 0.1 suggest poor algorithmic pickup or audience disinterest.

Exposure Efficiency (EE) normalizes audience interaction relative to reach, enabling fair comparison between high-view and low-view content. It's calculated as

$$EE_v = \frac{V_v}{S_c}$$

A video with 1,000 views and 100 interactions (10% engagement) demonstrates stronger audience resonance than a video with 100,000 views and 5,000 interactions (5% engagement), despite the latter's absolute magnitude.

Percentile Normalization. Recognizing that social media metrics follow power-law distributions, where means are dominated by outliers and most content clusters near zero, the system employs SQL window functions to transform raw metrics into normalized percentiles ranging [0,1]. This mathematical transformation enables meaningful cross-video comparisons by positioning each video within the performance distribution rather than relying on absolute values. A video at the 95th percentile for views has objectively outperformed 95% of the dataset, regardless of whether the absolute view count is 10,000 or 1 million. The resulting transformed variables `exposure_efficiency_pct` (EE_v^p), `engagement_rate_pct`(ER_v^p), `view_percentile`(V_v^p) enable robust cross-video comparison by emphasizing relative performance rather than absolute magnitude.

4.3 Composite Scoring Models

To programmatically identify high-value content opportunities, the system implements a dual-scoring mechanism with distinct strategic objectives. The **Best Content Score (BCS)** identifies proven high-performers:

$$BCS_v = 0.4 \cdot ER_v^p + 0.3 \cdot EE_v^p + 0.3 \cdot V_v^p$$

This weighted formula prioritizes engagement quality (40%) while rewarding both algorithmic efficiency (30%) and absolute reach (30%). The highest-scoring videos represent content that successfully combines audience interaction, platform amplification and mass visibility—these are the established hits.

The **Opportunity Score (OS)** identifies underexposed high-performing content, commonly referred to as "hidden gems":

$$OS_v = 0.4 \cdot ER_v^p + 0.4 \cdot (1 - EE_v^p) + 0.2 \cdot (1.0 - V_v^p)$$

By incorporating inverse percentile terms for exposure efficiency and view count, the score penalizes already saturated content while prioritizing videos that exhibit strong engagement relative to their reach. This captures situations where intrinsic content quality exceeds current platform exposure, highlighting candidates with high replication and amplification potential. The system extracts the top 20% of videos by each score (with a minimum threshold of one video for small datasets), then constructs the "Interesting Videos" set as the union of both rankings. This dual-selection approach ensures capture of both proven winners and untapped opportunities, establishing a comprehensive "interesting performance" subset for comparative analysis. One example is the following in the figure 1:

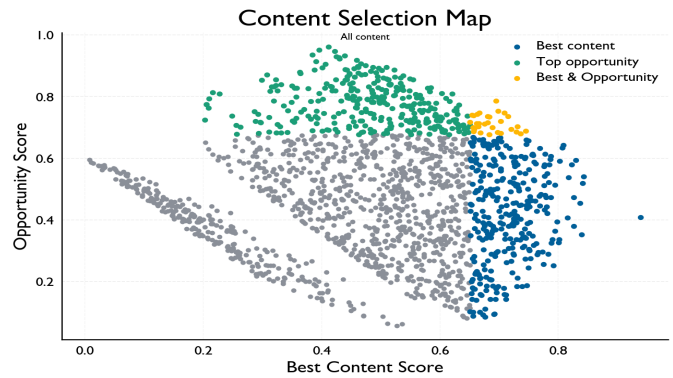

Figure 1: Content selection map - Opportunity vs Best score content

4.4 Google Trends Quality Pipeline

Google Trends data serves a different analytical role: it represents external demand rather than platform supply. Validation focuses on semantic normalization and temporal coherence rather than engagement consistency.

4.4.1 Normalization and Pattern Recognition

Both Top and Rising query datasets are cleaned by standardizing growth indicators. Percentage increases are converted into numeric values, while the special `breakout` label (indicating explosive growth exceeding 5000%) is mapped to a fixed high value to preserve ordering without distorting scale. A binary breakout flag is computed using a fixed threshold on growth percentage, explicitly encoding whether a query exhibits meaningful acceleration. The four binary indicators are concatenated in descending temporal order (60d, 30d, 14d, 7d) to form a 4-digit binary fingerprint that encodes the temporal activation profile of the query. For example, the pattern "1011" denotes a query that shows significant growth in the 60-day, 14-day and 7-day windows, while being absent in the 30-day window. This representation enables a compact and interpretable description of trend persistence, recency and recurrence across time.

4.4.2 Rising queries index

For Rising queries specifically, the system assigns higher weights to shorter temporal windows, reflecting the intuition

that growth observed over short horizons is more informative than growth observed over longer periods. A logarithmic transformation is applied to growth percentages before aggregation, damping extreme values while preserving relative ordering:

$$row_score = window_weight \times \log(1 + increase_pct)$$

where window weights exponentially favor recency: the 7-day window receives 8× weighting, 14-day receives 4×, 30-day receives 2× and 60-day receives 1×. Temporal patterns are ranked hierarchically, with specific patterns addressing highest priority as they indicate brand-new emerging trends from recurrent rising. This ranking system mathematically encodes editorial judgment about trend life cycle stages. The final ranking extracts the top 20% of queries based on pattern rank (primary) and recency score (secondary), with a minimum threshold of one query to prevent empty result sets for small datasets.

4.4.3 Ranking of Top Queries

The system focuses exclusively on short-term windows (7d, 14d) for top queries, pivoting multiple rows per query into single consolidated records that track presence flags and corresponding metrics. When a query appears in both windows, the 7-day window is prioritized as the more recent and actionable signal.

```sql
CREATE TABLE {GOOGLE_TOP_RANKED} AS
WITH ranked AS (SELECT *,CASE pattern
                          WHEN '11' THEN 1
                          WHEN '01' THEN 2
                          WHEN '10' THEN 3
                          WHEN '00' THEN 4
                          ELSE 99
                        END AS pattern_rank
                      FROM {GOOGLE_TOP_COMPUTED
                          }),
ordered AS (SELECT *,
ROW_NUMBER() OVER (ORDER BY pattern_rank ASC,
    final_score DESC) AS rn,
                      COUNT(*) OVER () AS
                          total_rows
            FROM ranked)
SELECT query
FROM ordered
WHERE rn <= CASE
  WHEN total_rows < 5 THEN 1
  ELSE CAST(0.2 * total_rows AS INTEGER)
END
ORDER BY rn;
```

The final score combines search volume and growth momentum:

$$final_score = 0.7 \cdot search_interest + 0.3 \cdot increase_pct$$

This 70/30 weighting reflects strategic priority on absolute search volume over growth rate for top queries, as these represent established rather than emerging trends. The top 20% are retained based on a hierarchical ranking where temporal pattern ($11 > 01 > 10$) takes precedence over the composite score.

The final ranked tables for both Top and Rising queries are materialized as physical SQL tables, representing the canonical demand-side inputs to the reasoning layer.

5 Data Analysis and Statistical Modeling

By positioning analytical scoring as a decision layer rather than a descriptive one, the system enforces a clear boundary between data-driven selection and generative synthesis. This design has three key implications. First, it improves explainability, as every generated suggestion can be traced back to a finite set of selected content items and explicit scoring policies. Second, it improves robustness by preventing noise and low-quality signals from propagating into the reasoning phase. Third, it aligns the system with a principle of controlled generation, where creativity is supported, but not replaced, by empirical evidence.

5.1 Architectural Principles

Three fundamental principles guide the analytical design. First, SQL-first processing ensures that all filtering, aggregation and window functions execute in the relational database engine, leveraging compiled performance rather than interpreted scripting loops. Second, separation of concerns maintains Python for orchestration and visualization, while SQL handles transformation and validation, enabling independent testing and optimization of each layer. Third, transparent isolation through dedicated tables and percentage reports makes cleaning decisions verifiable and reversible, supporting reproducible research and enabling stakeholders to inspect transformation logic at any stage of the pipeline.

5.2 The Double-Pass YouTube Analytical Protocol

Following SQL processing, the EDA module executes a comparative analysis framework designed to isolate success differentiators. The methodology employs two sequential passes over the dataset, each serving a distinct analytical purpose. The first pass analyzes the full cleaned dataset (`youtube_full_statistics`), computing comprehensive descriptive statistics, like mean, median, quartiles, 90th and 95th percentiles, for all content. This establishes the "average" benchmark, a market-wide performance norm against which exceptional performance can be measured. Without this baseline, it would be impossible to quantify what "good" performance actually means within a given competitive context. The second pass re-executes the identical analysis exclusively on the "Interesting Videos" subset, specifically those videos identified by the dual-scoring mechanism as either proven high-performers or hidden gems. By contrasting these elite performers against the global baseline, the system mathematically isolates specific performance deltas: differences in publishing timing, engagement patterns and content characteristics that differentiate viral success from average performance. Rather than simply describing top performers, it quantifies the magnitude of difference between elite and average content, transforming qualitative observations ("this video performed well") into actionable numerical targets ("elite videos publish on Wednesdays at 3 PM and achieve 2.3 times higher engagement rates").

5.2.1 Advanced Segmentation and Temporal Analysis

Beyond central tendency statistics, the module implements multi-dimensional stratification. Content is automatically classified into three performance tiers (Low, Medium, High) using equal-frequency binning. This segmentation enables compositional analysis, for instance, determining whether Reels are slightly more represented in the High tier compared to their overall dataset proportion, a key indicator of format-specific algorithmic favourability. An example is provided for the figure 2:

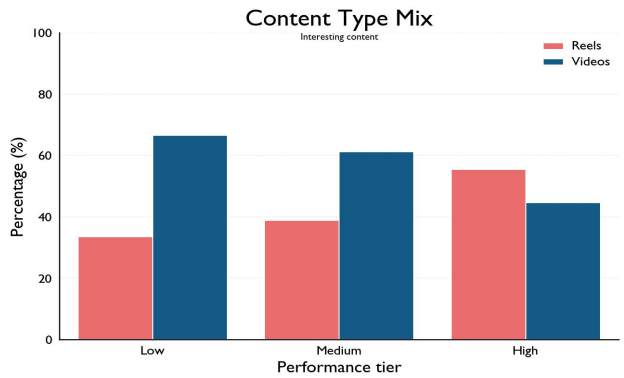

Figure 2: Content Type Mix by performance tier

Median view counts (rather than means, to resist outlier influence) are aggregated by day-of-week (figure 3) and hour-of-day (figure 4), identifying statistically optimal publication windows. In the example provided by the following figures, could be seen that the best days to public reels (Thursday) are different from the ones to public videos (Monday), other than the publication timing (reels perform better during the late evening) for each specific case.

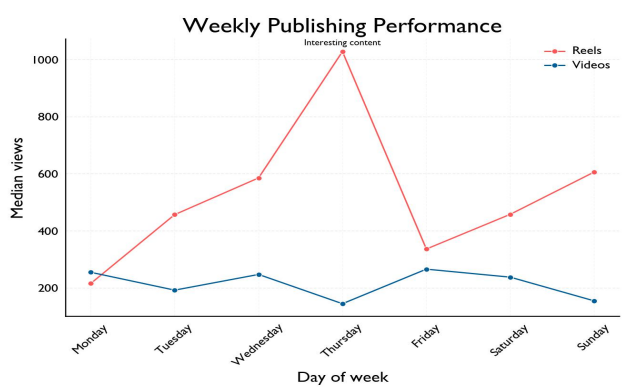

Figure 3: Daily performance trends

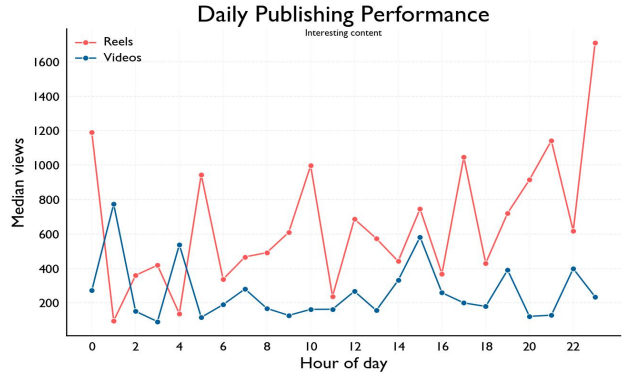

Figure 4: Hourly performance trends

These temporal patterns are materialized as dedicated trend tables and visualizations, providing creators with concrete scheduling recommendations grounded in observed performance data.

5.2.2 Inferential Statistics and Hypothesis Testing

To validate that observed performance differences between content formats are statistically significant rather than random variance, the system employs rigorous hypothesis testing. Welch's t-test is applied to compare Videos and Reels across all performance metrics. This test was selected over Student's t-test precisely because it does not assume equal variance, a critical consideration when comparing the volatile performance distribution of Reels against the more stable distribution of long-form Videos. Beyond p-values, the system computes percentage differences in means:

$$diff_pct = \frac{mean_{Reels} - mean_{Videos}}{mean_{Videos}} \times 100$$

This provides actionable context: a statistically significant difference with only 5% effect size carries different strategic implications than a 200% difference. The combination of significance testing and effect size quantification ensures that recommendations are both statistically valid and practically meaningful.

5.2.3 Automated Reporting and Visualization

The pipeline implements a dedicated function that produces human-readable synthesis documents containing dataset composition breakdowns, complete descriptive statistics tables, t-test results with significance annotations, temporal optimization recommendations (best days/hours to post), performance tier distributions and top-5 performer listings for both Videos and Reels.
Reports are generated for both the complete dataset and the interesting subset, enabling side-by-side comparison.
The system generates analytical plots on logarithmic scales to accurately represent the power-law nature of social media metrics. These include view distribution histograms with kernel density estimation overlays, box plots comparing Videos versus Reels, line plots showing median views by day-of-week with format-specific trends, hour-of-day analyses identifying peak audience activity windows and stacked bar

charts showing Reels-to-Videos ratios in Low/Medium/High performance segments. In the following example, the figures 5 and 6, it could be seen that the distribution itself is different from the reels to the videos showing the favourability of the algorithm.

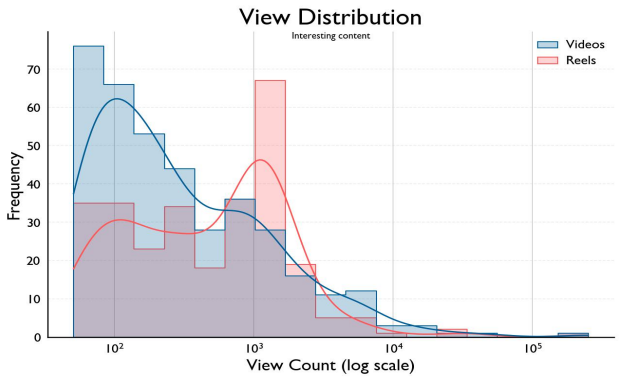

Figure 5: View distribution histogram on log scale by type

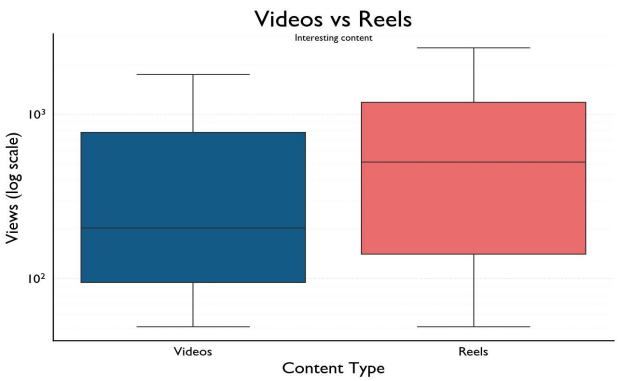

Figure 6: View distribution boxplots on log scale by type

6 LLM-Based Reasoning Layer

The LLM-based reasoning layer represents the final computational component of the system. Its role is not to perform discovery or evaluation, but to synthesize ideation outputs from a constrained and pre-validated input space. In the proposed architecture, the LLM operates downstream of all data acquisition, quality enforcement and analytical decision-making. By the time data reaches the LLM, the system has already reduced uncertainty, enforced consistency and selected a finite set of relevant signals.

6.1 Separation Between Decision and Generation

The analytical layer has already determined which videos represent proven success patterns, which queries represent rising demand and which temporal windows optimize visibility. So the LLM receives a curated representation of the decision layer output and is asked to transform this information into human-interpretable suggestions. This separation significantly reduces the risk of hallucination and aligns the generative output with empirical evidence rather than speculative reasoning.

6.2 Prompt Design as an Architectural Component

The prompt used to interact with the LLM is treated as an integral part of the system architecture and explicitly encodes the system's priorities and constraints. Within the prompt, performance, the LLM is instructed to align suggestions with observed title formats, tag usage and temporal patterns from high-performing content. Meanwhile, search demand signals influence framing and emphasis, rising queries inform topical focus and keyword selection. The prompt also specifies the expected output format, limiting the scope of generation to predefined elements: proposed video titles, recommended tags, descriptive keywords and optimal publication timing.

6.3 Structured and Constrained Generative Output

The LLM output consists of a finite set of concrete content proposals derived from the validated analytical context. For each execution, the system generates exactly three recommendations for long-form videos and three for short-form content (reels), each representing a distinct yet data-consistent editorial angle.

Each recommendation is expressed as a structured content specification, including a proposed title, search-aligned tags and keywords and a suggested publication time. Within each content type, the proposals are intentionally non-redundant, differing in framing, emphasis or secondary perspective while sharing a common analytical foundation.

In addition to individual proposals, the system exposes a high-level content type decision indicating whether long-form videos, short-form reels or a balanced combination should be prioritized. This decision is computed upstream by the analytical layer based on empirical performance and demand signals and provided to the generative layer as an explicit contextual constraint.

As a consequence, the LLM does not autonomously choose between content formats but adapts its output to a pre-established strategic direction, conditioning generation on both topical relevance and format-level prioritization resolved by deterministic logic.

By constraining both the number and structure of generated proposals, the system avoids open-ended ideation and delivers a small, actionable set of alternatives suitable for editorial planning, while preserving traceability between analytical evidence and generative suggestions.

LLM CONTENT STRATEGY DECISION

- **Preferred content type:** videos
- **Confidence level:** medium
- **Justification:** Among the three metrics tested, only `comment_count` is statistically significant with `p_value` = 0.0041.

The `diff_pct` for `comment_count` is -57.1%, which favors videos.

The other two metrics are not statistically significant, so they do not provide strong evidence.

Since the only significant metric favors videos, it is justified to prioritize videos with medium confidence due to limited significant evidence.

GENERATED OUTPUT: Data Science — VIDEOS

VIDEO 1
- **Title:** How to Crack AI Roles in IT in 2026 | Complete Roadmap & Skills for Data Scientist, ML Engineer, GenAI Engineer
- **Tags:** AI roles 2026, Data Scientist career, Machine Learning Engineer, GenAI Engineer roadmap, AI job preparation, Data Science career 2026
- **Keywords:** how to crack AI roles in IT 2026, data scientist career roadmap 2026, machine learning engineer skills 2026, genai engineer job preparation
Publish: Sunday at 01:00

VIDEO 2
- **Title:** 2026 Data Scientist Roadmap from Zero to Advanced | Complete Guide to Become a Top Data Scientist
- **Tags:** Data Scientist roadmap 2026, Become data scientist, Data science career guide, Advanced data science skills, Data science beginner to pro
- **Keywords:** how to become data scientist 2026, data scientist roadmap from zero, advanced data science skills 2026, data science career guide 2026
Publish: Tuesday at 15:00

VIDEO 3
- **Title:** Full Stack Data Science with Generative AI & Agentic AI Tutorials | 12 Weekends Batch Complete Course 2026
- **Tags:** Full stack data science 2026, Generative AI tutorials, Agentic AI course, Data science full course, AI tutorials 2026
- **Keywords:** full stack data science 2026, generative AI tutorials 2026, agentic AI data science course, complete data science course 2026
Publish: Wednesday at 23:00

GENERATED OUTPUT: Data Science — REELS

REEL 1
- **Title:** The Struggle of Live Tech Interviews | Data Science Memes #shorts
- **Tags:** datascience, interview, techmemes, careeradvice, jobinterview, shorts
- **Keywords:** data science interview, tech interview struggles, data science memes, job interview tips, interview preparation
Publish: Sunday at 23:00

REEL 2
- **Title:** El mayor error en Data Science con IA | Avoid This Common Mistake!
- **Tags:** datascience, artificialintelligence, machinelearning, shorts, ai, mistakes
- **Keywords:** data science mistakes, artificial intelligence errors, machine learning tips, common AI errors, data science with AI
Publish: Wednesday at 0:00

REEL 3
- **Title:** Deloitte Data Analyst Certification | Boost Your Career in Data Analytics #shorts
- **Tags:** dataanalytics, dataanalyst, certification, datascience, careergrowth, shorts
- **Keywords:** Deloitte data analyst certification, data analyst certification, data analytics career, data science certification
Publish: Monday at 21:00

6.4 Controlled Generation and Explainability

By constraining the LLM's input space and explicitly defining its task, the system achieves a high degree of explainability. Every generated suggestion can be traced back to a finite set of selected content items, explicit scoring policies and a fixed prompt specification. Consequently, the LLM layer complements the analytical pipeline without undermining its determinism. Generation is treated as the final step of a controlled process where creativity is guided by validated data rather than replacing it. This design philosophy ensures that outputs remain both innovative and empirically defensible.

7 User Interface and Orchestration Layer

The final layer of the architecture is the user interface and orchestration component. Unlike traditional graphical interfaces focused on visualization, this layer is designed as a control surface for the entire pipeline. Its primary purpose is to coordinate execution, manage configuration parameters and expose the system's functionality in a controlled and reproducible manner. The interface is implemented as the entry point through which users interact with the data platform and the reasoning engine.

7.1 Orchestration Role and Execution Flow

The user interface does not perform data processing or analytical computations directly. Instead, it orchestrates the execution of the underlying modules by invoking acquisition, quality enforcement, scoring and reasoning in a predefined sequence. This enforces a clear separation between operational control and computational logic, avoiding hidden dependencies between modules and ensuring that each stage is executed under explicit user control. Users are not required to interact directly with low-level SQL scripts or API logic, yet all intermediate outputs remain accessible for inspection. This balance allows the system to be used both as a black-box ideation tool and as a white-box analytical pipeline. Advanced users can inspect intermediate tables and logs, while less technical users can rely on the interface to manage execution safely. The interface enforces a strict execution order that mirrors the architectural design: data acquisition is triggered first, followed by SQL-based quality enforcement and analytical scoring. Only after these deterministic steps are completed does the interface invoke the LLM-based reasoning layer. This sequential orchestration guarantees that generative reasoning is always grounded in the most recent and validated data, preventing partial or inconsistent executions that could lead to ambiguous or misleading outputs.

7.2 Parameter Management and Output Standardization

Through the interface, users can specify high-level parameters such as the target topic, storage paths and output directories. These parameters are propagated consistently across all pipeline stages, reducing the likelihood of configuration drift and enhancing reproducibility. Given the same inputs and parameters, the system produces identical outputs, independent of execution history. The interface is also responsible for exporting final artifacts generated by the pipeline,

including cleaned analytical tables, selected subsets of relevant content and structured outputs produced by the LLM. By standardizing output formats and storage locations, the interface ensures that results can be easily reused, shared or further analyzed outside the system, supporting integration with external tools while preserving the integrity of the internal workflow.

8 Limitations and Future Directions

Operational and Scalability Constraints The system's reliance on the YouTube Data API presents a scalability bottleneck. With a daily limit of 10,000 quota units per key, obtaining meaningful market coverage requires a pool of different distinct API credentials. The current architecture handles this via horizontal scaling (multiplying free-tier keys), but a production deployment would necessitate vertical scaling through a Compliance Audit and quota extension, transforming the challenge from credential management to cost management. Similarly, Google Trends data acquisition currently involves a manual CSV export step, introducing a "Human-in-the-Loop" dependency. This is a budgetary limitation rather than a technical one. Production systems could replace manual labor with Enterprise DaaS providers to enable automated, continuously refreshed data.

Analytical and Methodological Extensions The current analytical engine employs fixed weights and percentile thresholds, which ensures transparency but remains static across executions. Future developments could introduce adaptive weighting schemes or scenario-based profiles that adjust scoring parameters based on content category or competitive intensity. Another extension involves closing the feedback loop: the system currently generates suggestions based on historical data but does not "learn" from the actual performance of videos created using those suggestions. Reingesting post-publication metrics would transform the system from a static decision-support tool into a semi-adaptive platform capable of refining its own scoring parameters over time.

Generative Scope and Output Diversity The LLM is intentionally constrained to producing ideation primitives (titles, tags, keywords, timing) rather than full creative artifacts like video scripts. Future versions could include optional layers for script outlining or narrative structuring, provided that these expanded outputs remain strictly bound by upstream decision constraints to prevent hallucination and maintain empirical grounding.

9 Conclusions

This research demonstrates that content ideation can be effectively reframed as a data management and engineering problem rather than a purely creative endeavor. By enforcing a strict architectural separation between data acquisition, quality enforcement, analytical decision-making, and generative synthesis, the proposed system grounds ideation outputs in statistically validated evidence instead of speculative inference. Within this framework, empirically derived performance patterns act as structural guardrails for generative reasoning, ensuring that the resulting suggestions remain both innovative and defensible from a data-driven perspective. The system architecture further illustrates how a local, SQL-first data platform can support creator economy applications in a reproducible manner. The combination of a dual-score analytical framework, capturing both established performance patterns and underexposed opportunities, and the LLM as a synthesis engine, enables a comprehensive exploration of the content landscape while substantially reducing hallucination risk. Full traceability from raw API ingestion to final recommendations, achieved through materialized intermediate tables, ensures transparency and reproducibility across the entire pipeline. Implementation details, data schemas, and operational workflows are documented in a dedicated technical guide, provided in the file `Technical Guide`, which offers a complete step-by-step description of the system design.

10 References

The project makes reference to official documentation and platform policies provided by external services:

- **YouTube Data API Documentation**: Official reference for supported query parameters, ordering strategies, quota constraints and response semantics. `https://developers.google.com/youtube/v3`
- **Google Trends Terms and Limitations**: Defines usage conditions and known limitations of Google Trends data, including constraints on normalization and temporal comparability. `https://policies.google.com/terms`
- **OpenAI API Documentation**: Reference documentation for the generative API used in the reasoning layer. `https://openai.com/it-IT/api/`
- **Use of AI Tools**: AI-based tools were used during development to support code refinement and debugging.
- All the images and results are the output of a run of the entire program having for topic "data science" run on 6th of February 2026